

[Home](#)[GitHub](#)

CLASSES

[ArticlesService](#)[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)

backend/src/app.js

```
1.  /**
2.   * @file Fichero principal de la aplicación Express.
3.   * @description Configura y arranca el servidor Express, define middlewares y rutas.
4.   * @requires http-errors
5.   * @requires express
6.   * @requires path
7.   * @requires cookie-parser
8.   * @requires morgan
9.   * @requires cors
10.  * @requires helmet
11.  */
12.
13.  var createError = require('http-errors');
14.  var express = require('express');
15.  var path = require('path');
16.  var cookieParser = require('cookie-parser');
17.  var logger = require('morgan');
18.  var cors = require('cors');
19.  var helmet = require('helmet');
20.
21.  var app = express();
22.
23.  // view engine setup
24.  app.set('views', path.join(__dirname, 'views'));
25.  app.set('view engine', 'pug');
26.
27.  app.use(logger('dev'));
28.  app.use(helmet());
29.
30.  /**
31.   * @name CORS_Configuration
32.   * @description Configuración de CORS para permitir peticiones desde el frontend.
33.   * @property {string} origin - El origen permitido para las peticiones.
34.   * @property {boolean} credentials - Indica si se permiten credenciales.
35.   * @property {number} optionsSuccessStatus - Código de estado para peticiones de éxito.
36.   */
37.  // Configuración de CORS
38.  const corsOptions = {
39.    origin: process.env.FRONTEND_URL || 'http://localhost:3000', // Frontend URL
40.    credentials: true, // Permitir envío de cookies y headers de autenticación
41.    optionsSuccessStatus: 200
42.  };
43.  app.use(cors(corsOptions));
44.
45.  app.use(express.json());
46.  app.use(express.urlencoded({ extended: false }));
47.  app.use(cookieParser());
48.
49.  const authRouter = require('./routes/auth.routes');
50.  const formularioRouter = require('./routes/formulario.routes');
51.  const registroRouter = require('./routes/registro.routes');
52.  const diarioRouter = require('./routes/diario.routes');
53.  const iaRoutes = require('./routes/ia.routes');
54.  const contactoEmergenciaRouter = require('./routes/contactoEmergencia.routes');
```

```

55.   const healthRouter = require('./routes/health.routes');
56.
57.   app.use('/api/auth', authRouter);
58.   app.use('/api/formulario', formularioRouter);
59.   app.use('/api/registro', registroRouter);
60.   app.use('/api/registros', registroRouter); // Añadido para aceptar l
61.   app.use('/api/diario', diarioRouter);
62.   app.use('/api/health', healthRouter);
63.   app.use('/api', iaRoutes);
64.   app.use('/api/contactos-emergencia', contactoEmergenciaRouter);
65.
66.   //app.use('/', indexRouter);
67.   //app.use('/users', usersRouter);
68.
69.   /**
70.    * @name GET_/
71.    * @description Ruta de prueba para comprobar que el servidor funcio
72.    * @param {object} req - Objeto de petición de Express.
73.    * @param {object} res - Objeto de respuesta de Express.
74.    */
75.   app.get('/', (req, res) => {
76.     res.send('Funciona')
77.   })
78.
79.
80.   /**
81.    * @name 404_Error_Handler
82.    * @description Middleware para capturar errores 404 (Not Found).
83.    * @param {object} req - Objeto de petición de Express.
84.    * @param {object} res - Objeto de respuesta de Express.
85.    * @param {function} next - Función para pasar al siguiente middlewa
86.    */
87.   // catch 404 and forward to error handler
88.   app.use(function(req, res, next) {
89.     next(createError(404));
90.   });
91.
92.   /**
93.    * @name Generic_Error_Handler
94.    * @description Middleware para gestionar todos los demás errores.
95.    * @param {object} err - Objeto de error.
96.    * @param {object} req - Objeto de petición de Express.
97.    * @param {object} res - Objeto de respuesta de Express.
98.    * @param {function} next - Función para pasar al siguiente middlewa
99.    */
100.  // error handler
101.  app.use(function(err, req, res, next) {
102.    // set locals, only providing error in development
103.    res.locals.message = err.message;
104.    res.locals.error = req.app.get('env') === 'development' ? err : {}
105.
106.    // render the error page
107.    res.status(err.status || 500);
108.    res.render('error');
109.  });
110.
111.  module.exports = app;

```